DMA (Direct Memory Access) alapú periféria kezelés

Scherer Balázs



Budapesti Műszaki és Gazdaságtudományi Egyetem (BME),
Mesterséges Intelligencia és Rendszertervezés Tanszék (MIT)

© BME-MIT 2026

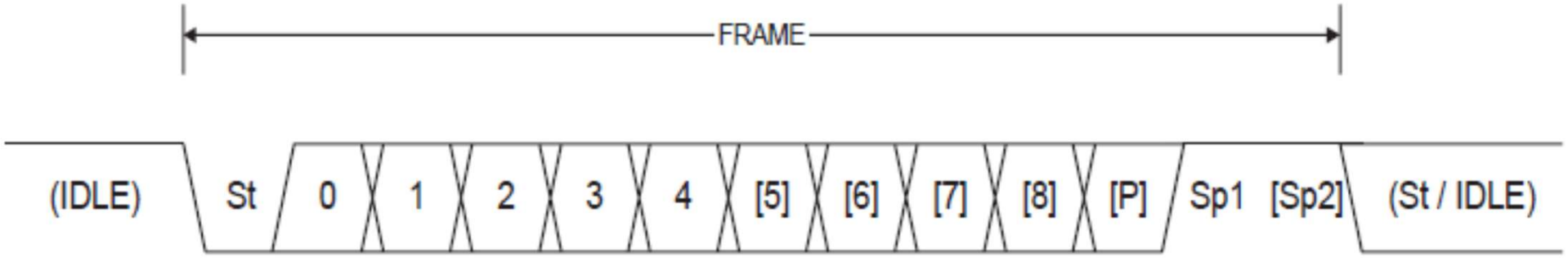
Tartalom

- **Az interruptos perifériakezelés korlátai**
- DMA alapú perifériakezelés
  - A DMA kezelés alapjai
  - A DMA általános konfigurációs lehetőségei
  - Példák DMA kezelésére
- Tipikus DMA kezelési hibák. Mit ne csináljunk
- A DMA helye a rendszer architektúrában
- DMA alapú perifériakezelés korlátai

© BME-MIT 2026

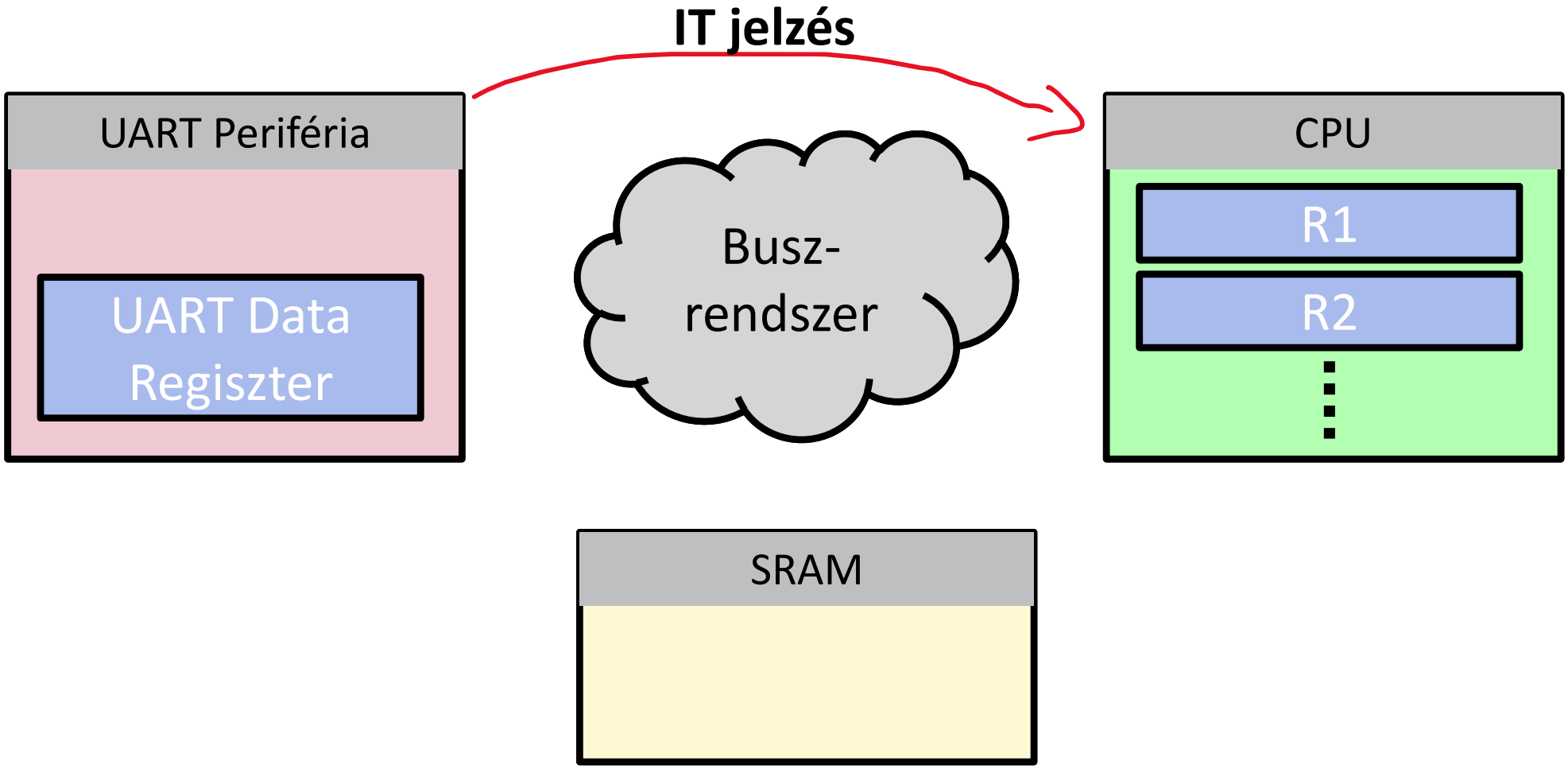
Gyors perifériák kiszolgálásának problémái

- Példa a gyors perifériák okozta problémákról:
  - UART 1Mbaud sebességű kommunikáció



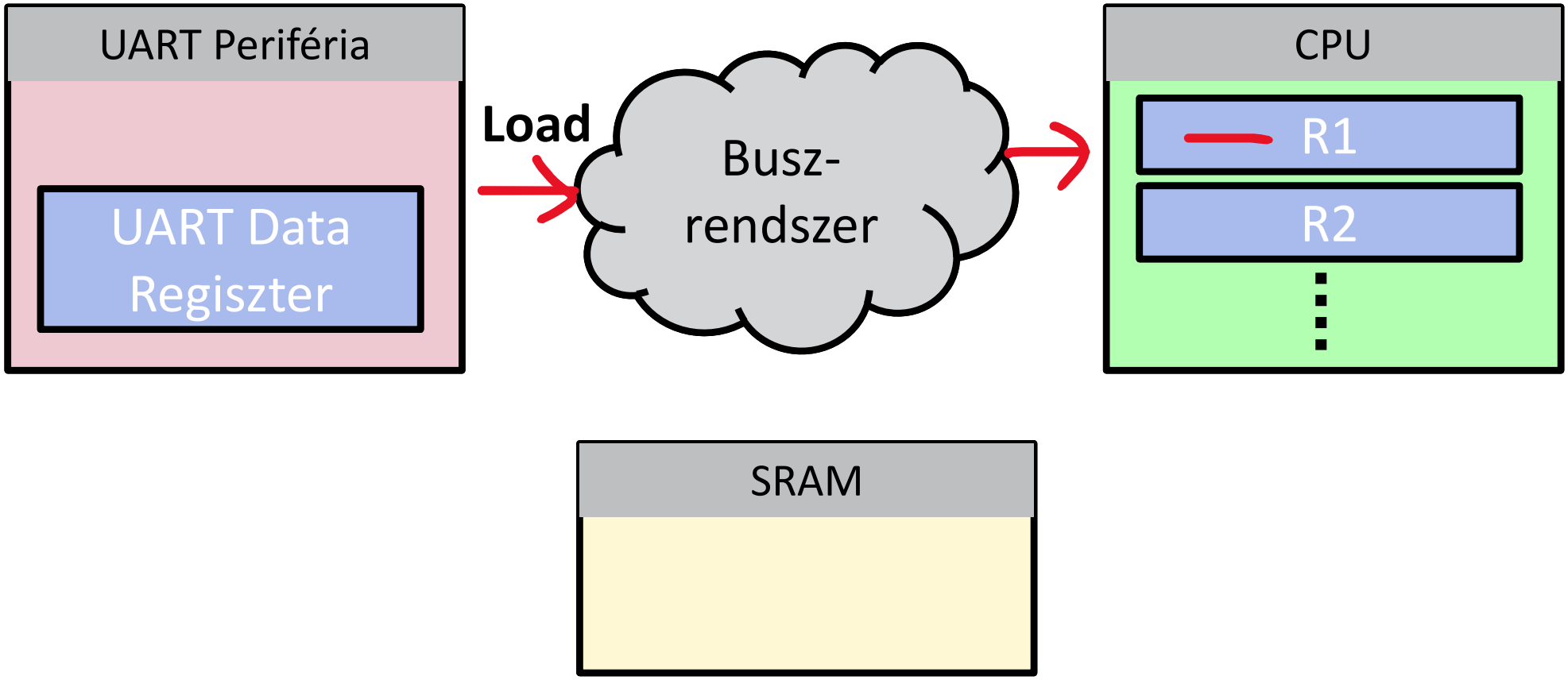
  - 1 start bit, 8 adatbit, 1 stop bit => nettó maximális adatátviteli sebesség 100kbyte/sec
  - 10µs-onként jön egy byte

UART átvitel megszakítás alapon



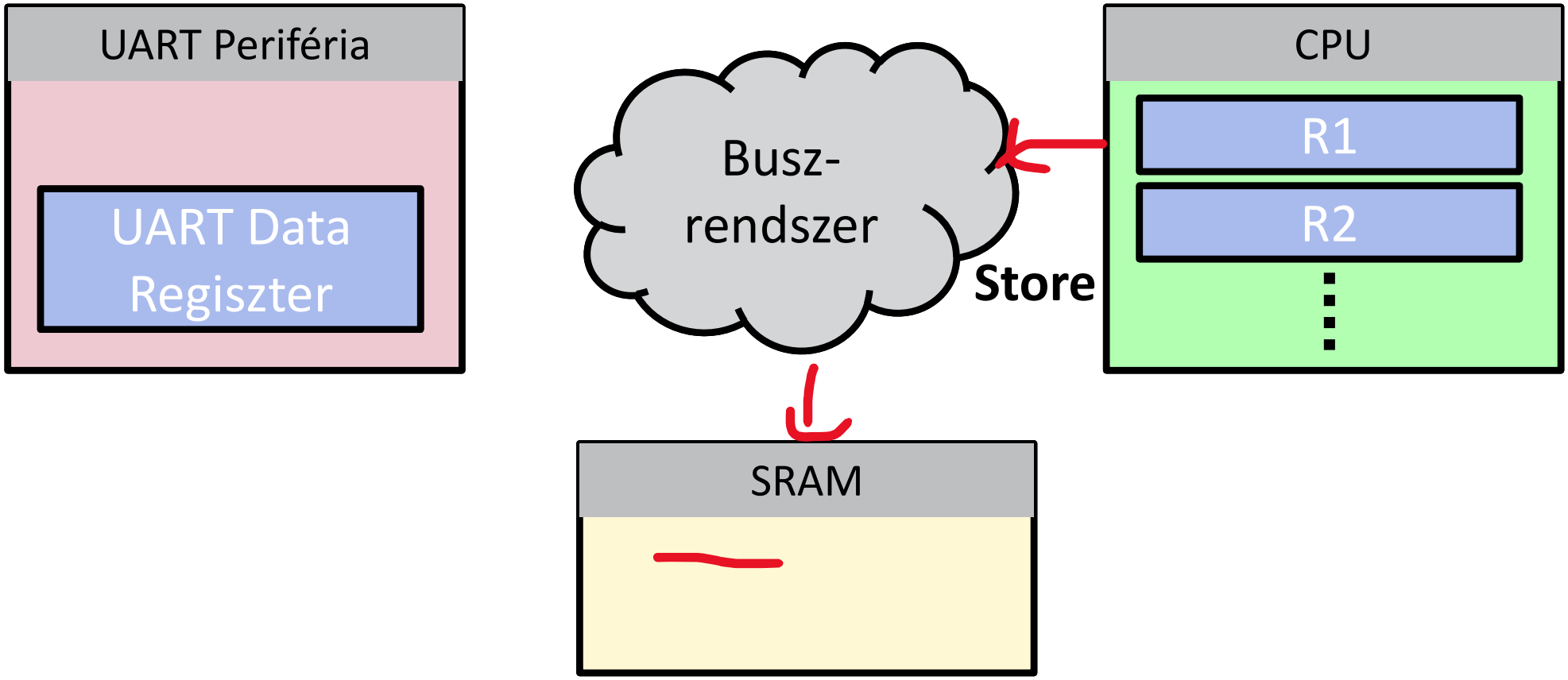
- 1. lépés az UART interruptot jelez

UART átvitel megszakítás alapon



- 2. CPU elkezdi az IT kiszolgálást
- 3. A CPU beolvassa az UART adatát (Load and Store architektúra: közvetlen nem tud memóriából memóriába mozgatni adatot)

UART átvitel megszakítás alapon



- 4. A CPU csinál valamit az adattal, de tipikusan mindenképpen legalább letárolja azt
- 5. Store: adat tárolás

Mennyi időbe kerül is ez nekünk?

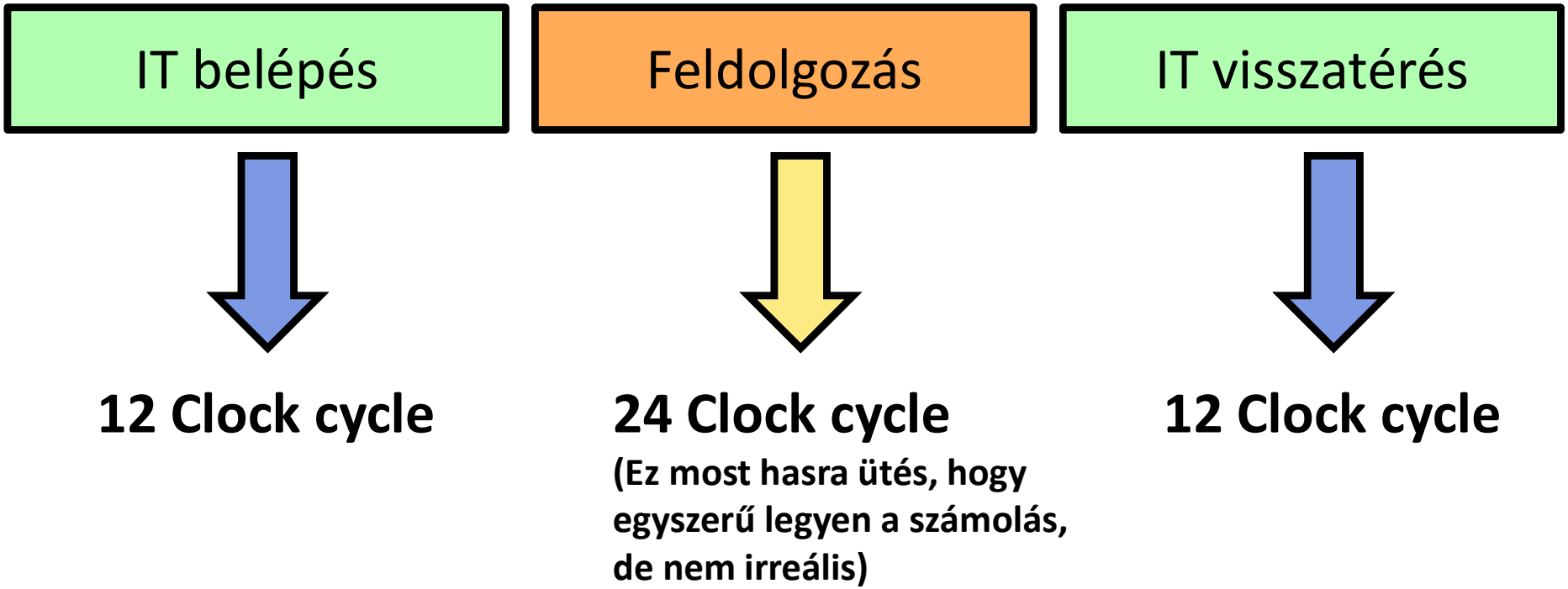
IT belépés

Feldolgozás

IT visszatérés

© BME-MIT 2026

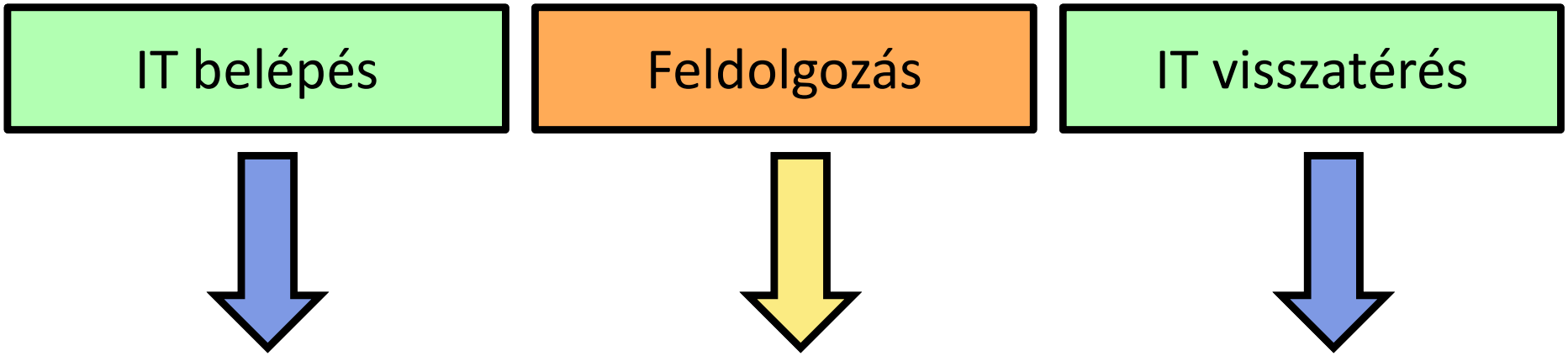
Mennyi időbe kerül is ez nekünk?



- Az általunk használt vezérlőnek 48MHz-es max. sebessége van
- 48 óraciklus az UART IT kiszolgálás => **IT kiszolgáláshoz szükséges idő 1μs**

© BME-MIT 2026

Mennyi időbe kerül is ez nekünk?



12 Clock cycle

24 Clock cycle
(Ez most hasra ütés, hogy egyszerű legyen a számolás, de nem irreális)

12 Clock cycle

- **Az említett 1MBaud sebességű kommunikáció kezelése 10%-os processzor load-ot okoz**
- **Ebből a 10%-ból minimális a hasznos rész (ebben a példában ~2% lehet)**

Egyéb példák gyors perifériák kiszolgálásának problémáira

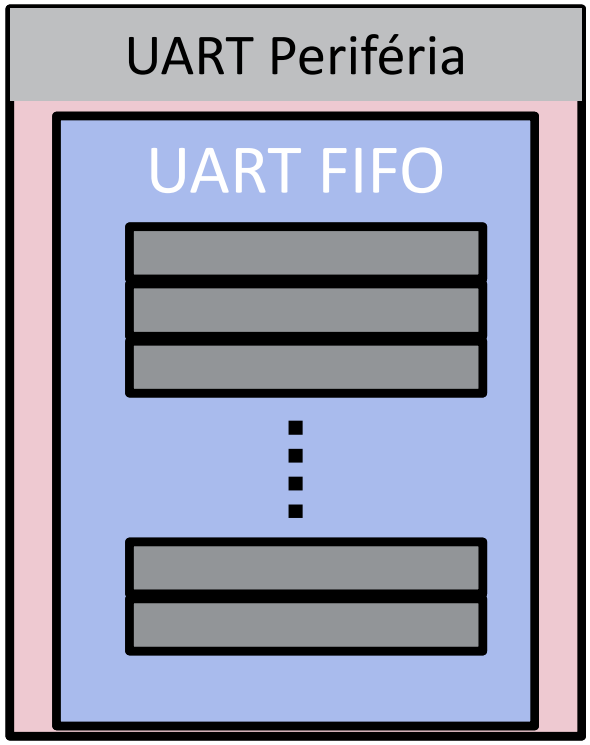
- Gondoljunk bele, hogy ez egy 1Mbaud/sec-es kommunikáció volt

- Mi lenne, hogyha mondjuk több ADC csatornát kellene mintavételezni n*100kHz-el?
- USB Full-speed kommunikáció „csak” 12Mbit/sec-el?
- Ilyen perifériákat párhuzamosan is kell néha kezelni
- Ne felejtsük el, hogy fent csak általános processzor terhelési overheadet írtunk, de van deadline is, tehát az UART-nál a byte elvételét a következő byte érkezése előtt meg kell csinálni.

Mit lehet tenni, hogy megoldjuk ezt a problémát?

© BME-MIT 2026

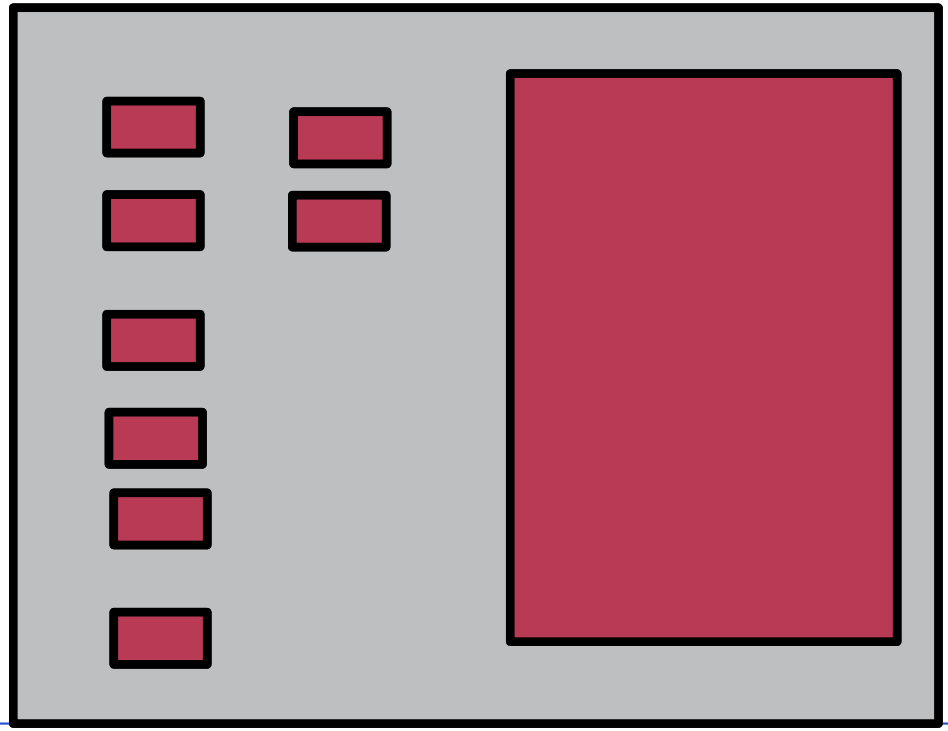
Tradicionális megoldás: lokális bufferek alkalmazása



- Folyamatos kommunikáció esetén elég csak ritkábban üríteni a FIFO-t
- Pl.: 16byte-os FIFO esetén arányosan csökkenhet legalább az IT overhead

Tradicionális megoldások hátránya

- Egy modern mikrovezérlőben rengeteg gyors periféria található (n*10 darab)
- Az ilyen FIFO-k technológiailag nagy chip felületet igényelnek (akkor is kellenek, ha nem használjuk őket)



Tradicionális megoldások hátránya

- Egy modern mikrovezérlőben rengeteg gyors periféria található (n*10 darab)
- Az ilyen FIFO-k technológiailag nagy chip felületet igényelnek (akkor is kellenek, ha nem használjuk őket)
- Minden olyan, ami adatot tárol jelentősen növeli az energia takarékos módok fogyasztását
- Van ezek mellett egy elég nagy általános SRAM, amibe utána úgyis átmásolunk mindent (később speciális esetekben visszajönnek a FIFO-k is, hogy a DMA hatékonyságot növeljék)
- *(Ettől függetlenül a nagyon gyors perifériáknál: USB, Ethernet jellemzően megmaradnak a helyi FIFO-k is)*

Összefoglalás: A megszakításos perifériakezelés korlátai

- A nagy kommunikációs sebességet, adatátvitelt igénylő perifériák, (nagy sebességű UART, ADC, DAC, SPI stb.) megszakításos kiszolgálása nagy rendszerterhelés okoz, mert önmagában az IT-be belépés és kilépés is jelentős számú órajelet igényel.
- Az esetek többségében a periféria események által kiváltott megszakítások után adatot kell mozgatni, feldolgozás csak később következik be.
- A gyakori megszakítások száma hardware-esen a mikrovezérlőben implementált lokális FIFO-kal csökkenthetők, de ez sok szempontból nem optimális.



Tartalom

- Az interruptos perifériakezelés korlátai
- **DMA alapú perifériakezelés**
  - **A DMA kezelés alapjai**
  - A DMA általános konfigurációs lehetőségei
  - Példák DMA kezelésére
- Tipikus DMA kezelési hibák. Mit ne csináljunk
- A DMA helye a rendszer architektúrában
- DMA alapú perifériakezelés korlátai

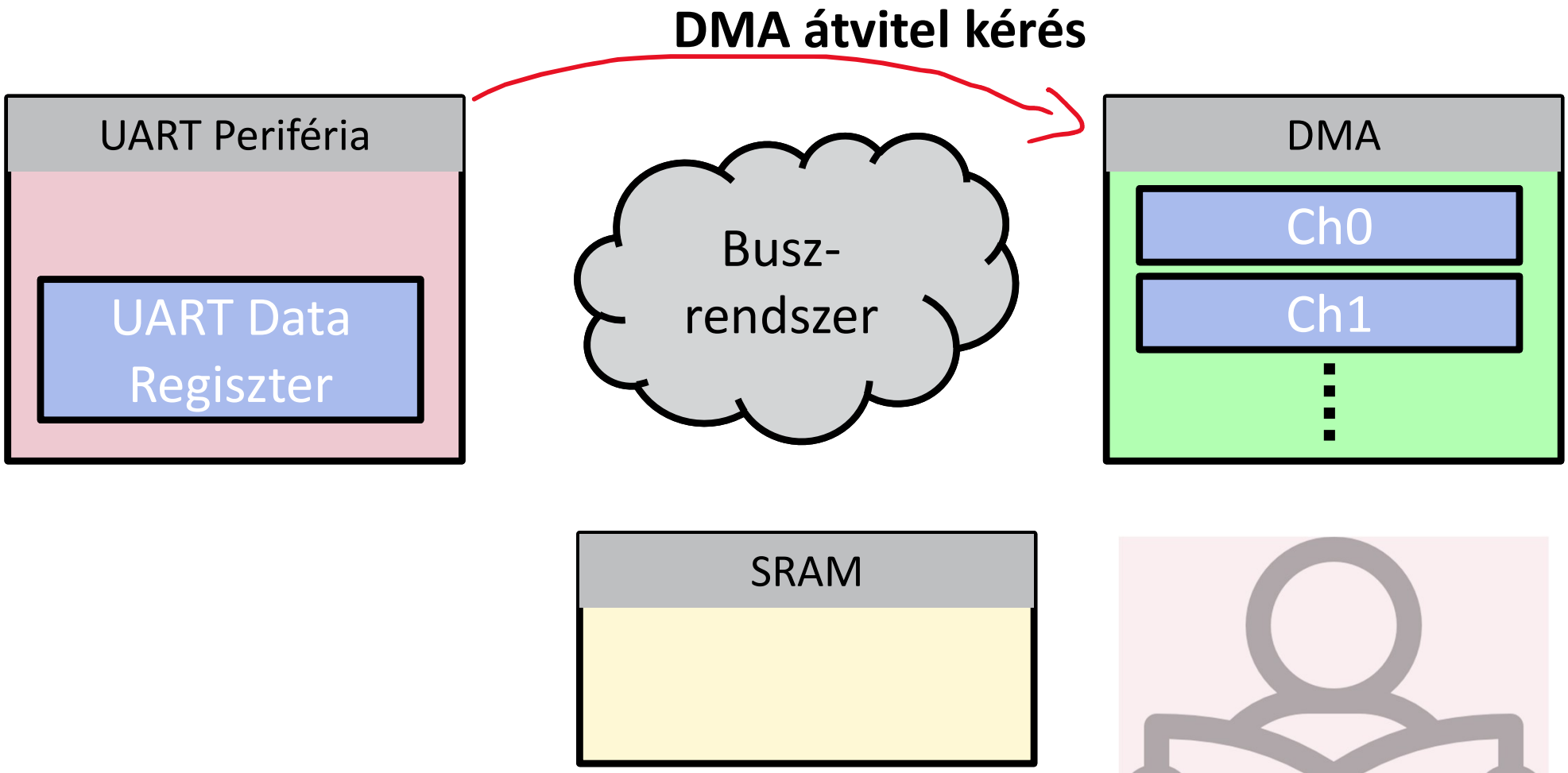
© BME-MIT 2026

Mi lenne, ha lenne egy olyan primitív „vezérlőnk”, ami a Load and Store műveletet eljátssza a háttérben az adott perifériára?

Hívjuk ezt a blokkot DMA-nak (Direct Memory Access)

© BME-MIT 2026

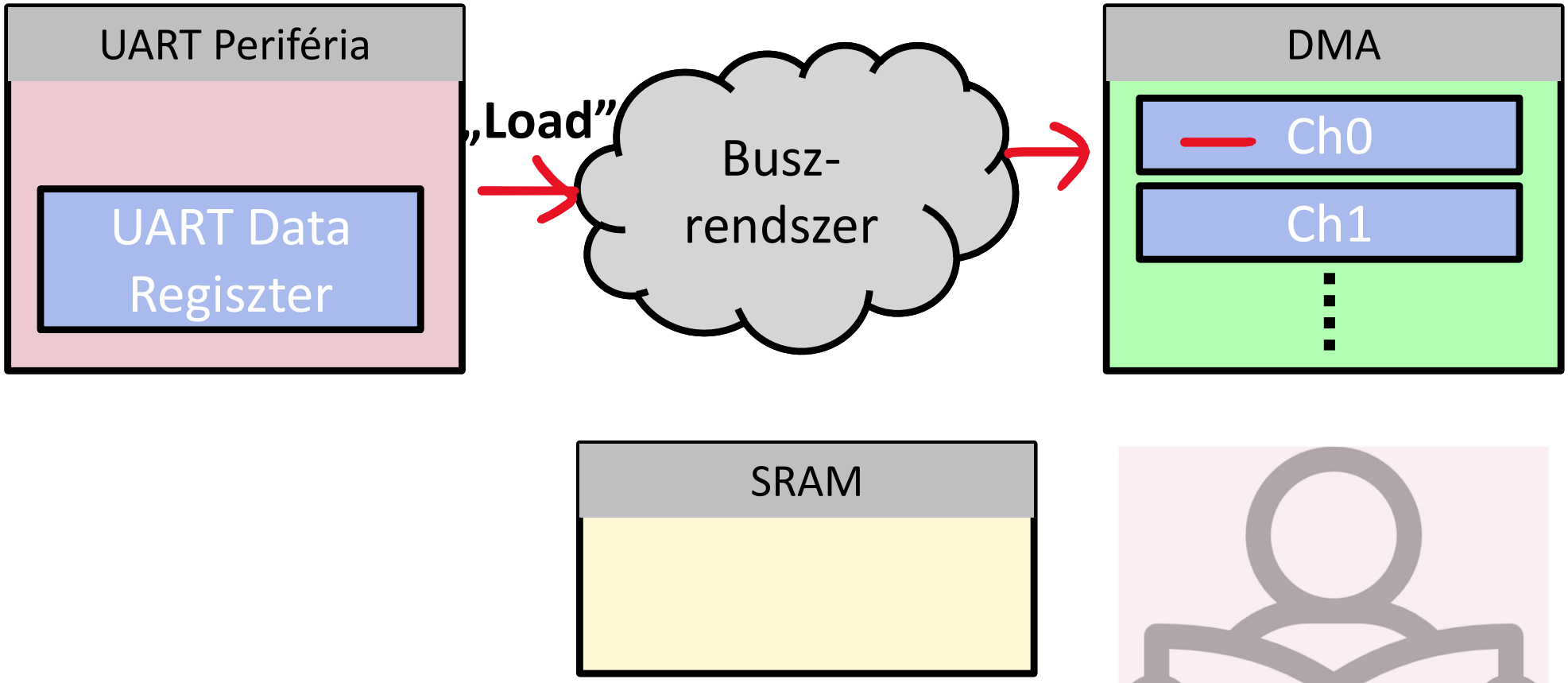
UART átvitel DMA alapon 1.



- 1. Az UART DMA átvitelt kér az új byte érkezésekor

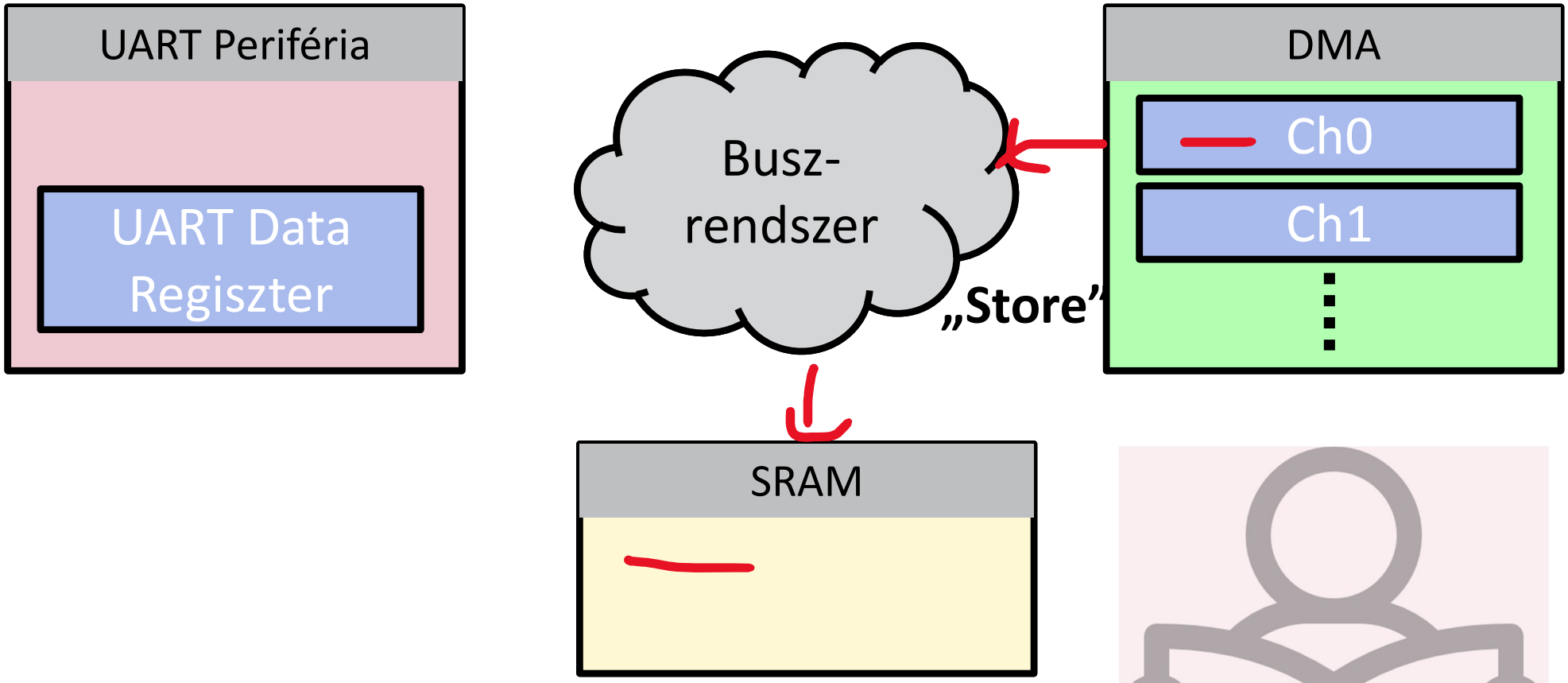


UART átvitel DMA alapon 2.



- 2. DMA a buszrendszeren keresztül beolvassa az adatot magába

UART átvitel DMA alapon 3.





- 3. A DMA kiírja az adatot a memóriába

DMA működés: általános tulajdonságok

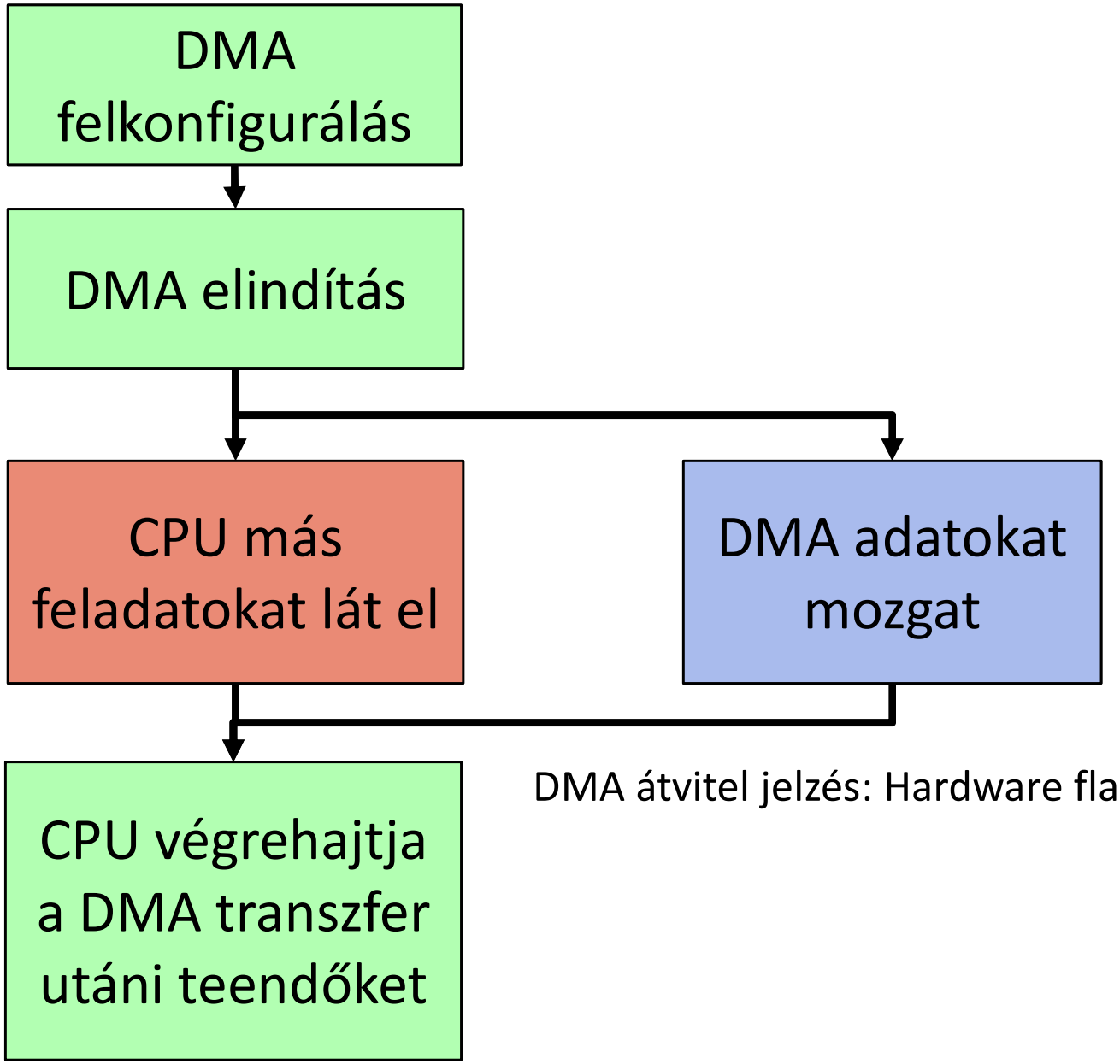
- **4 alap változata létezik**
  - **Memória-Memória**
  - **Memória-Periféria**
  - **Periféria-Memória**
  - **Periféria-Periféria**

- Minden DMA ciklus tipikusan legalább két busz ciklust igényel (nem 2 órajelet!)
  - Periféria/memória olvasást
  - Memória/Periféria írást

  (Ez a buszrendszer függvényében lényegesen bonyolódhat)
- A DMA vezérlő semmilyen feldolgozást nem végez az adatokon
- A DMA vezérlőt fel kell programozni
- A perifériát is be kell állítani, hogy a DMA kéréseket küldjön



DMA áttekintés: működés a processzorral együtt



DMA átvitel jelzés: Hardware flag vagy interrupt

Tartalom

- Az interruptos perifériakezelés korlátai
- **DMA alapú perifériakezelés**
  - A DMA kezelés alapjai
  - **A DMA általános konfigurációs lehetőségei**
  - Példák DMA kezelésére
- Tipikus DMA kezelési hibák. Mit ne csináljunk
- A DMA helye a rendszer architektúrában
- DMA alapú perifériakezelés korlátai

© BME-MIT 2026

DMA vezérlő programozása (összefoglalás)

- Átvitel forrás báziscíme (lehet memória vagy periféria cím)
- Cél báziscíme (lehet memória vagy periféria cím)
- Átviendő blokk hosszának beállítása
- Az adatszélesség forrás- és céloldalon (modern vezérlőknél eltérhet)
- Ciklus végéhez tartozó megszakítás jelzés
- Burst-ös (blokkos), vagy egyciklusú hozzáférés
  - Burst: hatékonyabb de lassabb reakció külső eseményekre
- Cirkuláris, vagy sima átviteli mód
- DMA csatorna/Stream megadása
- Prorítás megadás
- A legtöbb DMA kontroller megvalósítás képes arra, hogy a cél és/vagy a forrás címeket automatikusan inkrementálja.



Tartalom

- Az interruptos perifériakezelés korlátai
- **DMA alapú perifériakezelés**
  - A DMA kezelés alapjai
  - A DMA általános konfigurációs lehetőségei
  - **Példák DMA kezelésére**
- Tipikus DMA kezelési hibák. Mit ne csináljunk
- A DMA helye a rendszer architektúrában
- DMA alapú perifériakezelés korlátai

© BME-MIT 2026

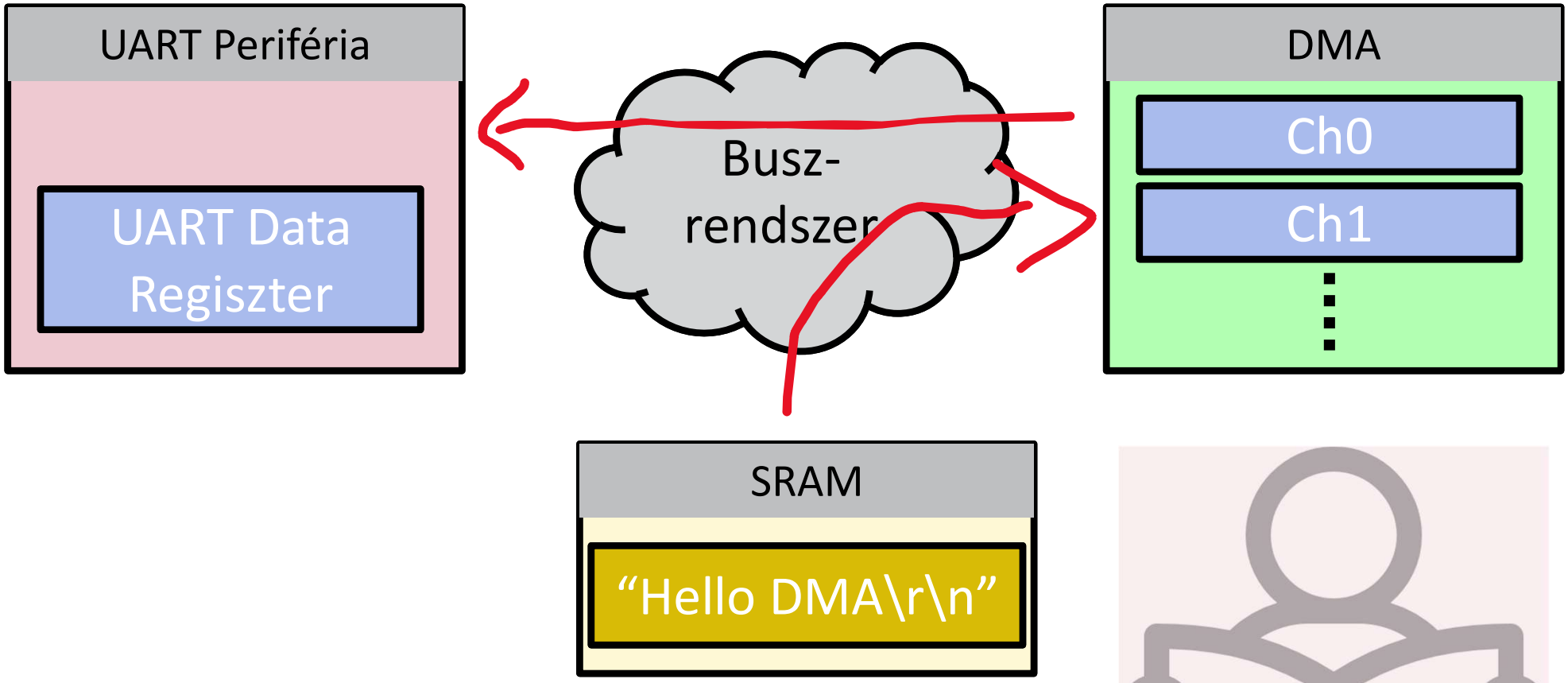
Mikor éri meg DMA-t használni

- Nagy mennyiségű burst-os adatátvitel esetében sokkal hatékonyabb, mint az interruptos verzió:
  - Kommunikációs csomagok átvitele
    - Hosszabb UART konzol üzenet
    - Lap írása nemfelejtő memóriába (SPI, I2C): 128byte, 256byte,512byte, vagy akár 2kbyte-os lapok
    - Nagy sebességű kommunikáció USB High-speed, Ethernet gyakorlatilag nem használható DMA nélkül, ezeknek sokszor dedikált beépített DMA vezérlőjük van.
- Háttérben zajló periodikus átvitelek
    - Pl.: 4 ADC csatornán folyamatosan mérünk: nem kell ennek túl gyorsnak lenni, lehet csak 1KHz-es, de folyamatos mérés esetén sokat lehet spórolni
    - LCD kijelzés vezérlés is gyakorlatilag nem működhet DMA nélkül (szintén sokszor dedikált 2D DMA): (320x240-es display, pixelenként 24bit infó, 60Hz-el => ~13Mbyte/sec átvitel)

© BME-MIT 2026

Egyszerű UART átvitel példa a DMA felkonfigurálásával

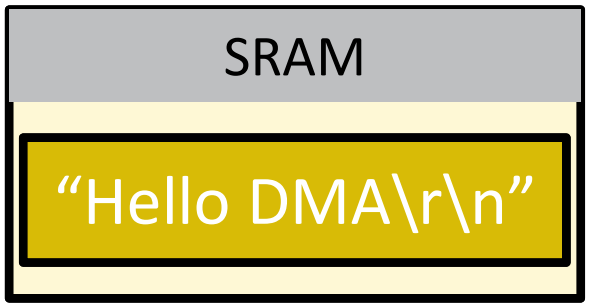
DMA felkonfigurálása: példa



- UART-on keresztül szeretnénk elküldeni egy stringet
  (lehetne ennél hosszabb üzenet is, hogy igazán megérje a dolog)

© BME-MIT 2026

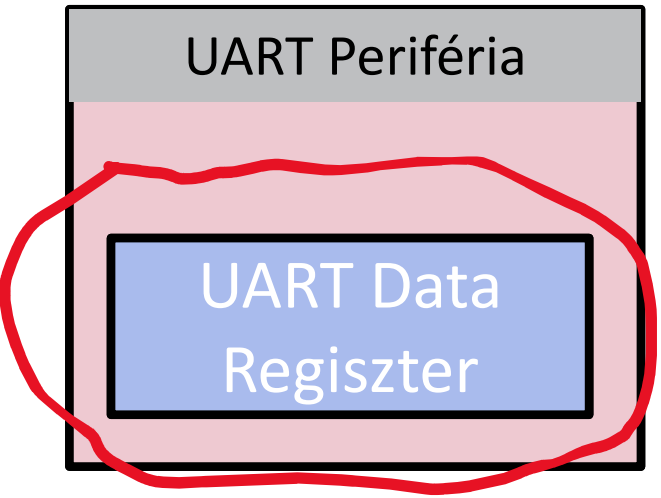
Forráscím, ebben az esetben memória cím



- Stringet globális változóban tároljuk, változó címeként elérhető a forrás címe
- Memória címeket általában inkrementáljuk

```
char msg[32] = ”Hello DMA \r\n”
sourceAddress = msg;
NumbOfByte = strlen(msg);
DataWidth = BYTE;
```

Célcím, ebben az esetben periféria cím



- Periféria címek ugyanúgy a memória térképbe vannak leképezve
- Általában direktben hivatkozni kell az adott adatregiszterre: CMSIS device.h

  &(UART0->TXDATA)

- Periféria címeket szinte soha nem inkrementálunk

© BME-MIT 2026

Alkalmazási példa:

Flash memória kezelés DMA-val

© BME-MIT 2026

MX25L6406 Flash chip tulajdonságai

- 64M-BIT memória méret (8 Mbyte)
- SPI interface, max. 80MHz-es órajel (mikrovezérlő általában nem tud ennyit)
- 256byte-os page-ek (a jellemzően egyszerre programozott memória méret)
- 4Kbyte-os Sector-okban, vagy 64Kbyte-os blokkokban, vagy az egész egyben törölhető



MX25L6406 Flash írás DMA beállítás

- **Átvitel forrás báziscíme:** a letárolandó memória tömb
- **Cél báziscíme:** az SPI periféria adatregisztere
- **Átviendő blokk hossza:** 256byte
- **Ciklus végéhez tartozó megszakítás jelzés:** igény szerint
- **Burst-ös, vagy egyciklusú hozzáférés:** általában burst-ös, de szituáció függő
- **Cirkuláris buffer, vagy sima átviteli mód:** sima átviteli mód
- **DMA csatorna/Stream megadása:** mikrovezérlő függő
- **Cél cím inkrementálás:** nincs
- **Forrás cím inkrementálás:** van



Alkalmazási példa:

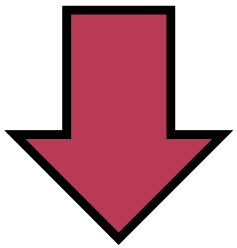
AD átalakító

Hogyan kell folyamatos mérést, feldolgozást megvalósítani?

© BME-MIT 2026

Cirkuláris buffer kezelés

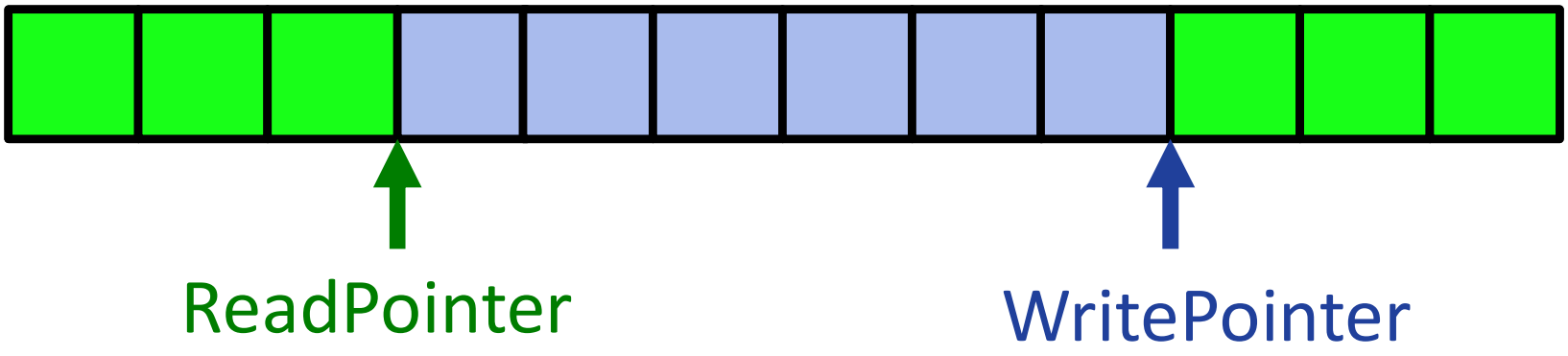
- **Nagyon sok esetben a beágyazott rendszerünk folyamatos mérést csinál**
  - Pl. AD átalakítást végzünk, az AD átalakítás után pedig az adatokat blokkosan időnként burst-ösen feldolgozzuk a főprogramban: (Szűrjük, átlagoljuk, felhasználjuk stb.)
  - Azt nem szeretnénk, hogy minden minta után külön kellene foglalkoznunk a feldolgozással (processzor idő)
  - Nincs végtelen helyünk a minták tárolására



- Ugyanazt a buffert kell folyamatosan felhasználnunk és tudnunk kell, hogy hol vannak a már feldolgozott és a még nem feldolgozott adatok.

Alap elgondolás folyamatos átvitelnél **szoftveres megvalósítással**

- **Read és Write pointerek használata**
  - Write pointer megy elől, addig követjük a read pointerrel, amíg utol nem érjük
  - A fordulás kezelése problémás



Feldolgozott/üres adatok

Friss, feldolgozatlan adatok

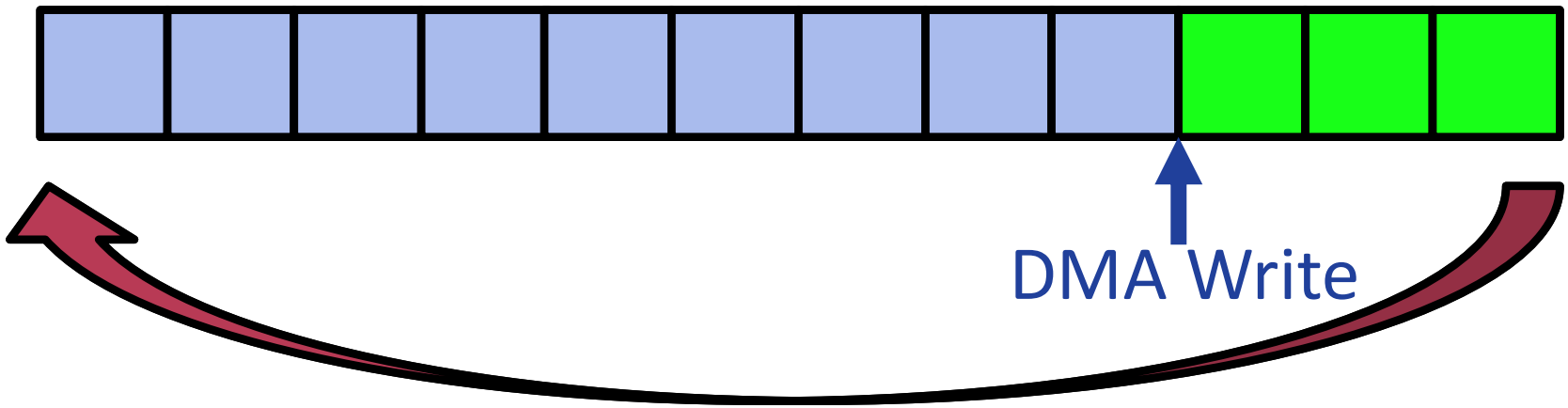
Cirkuláris buffer kezelés

- Minta Cirkuláris buffer kezelésre kóddal, és annak magyarázatával

https://embedjournal.com/implementing-circular-buffer-embedded-c/

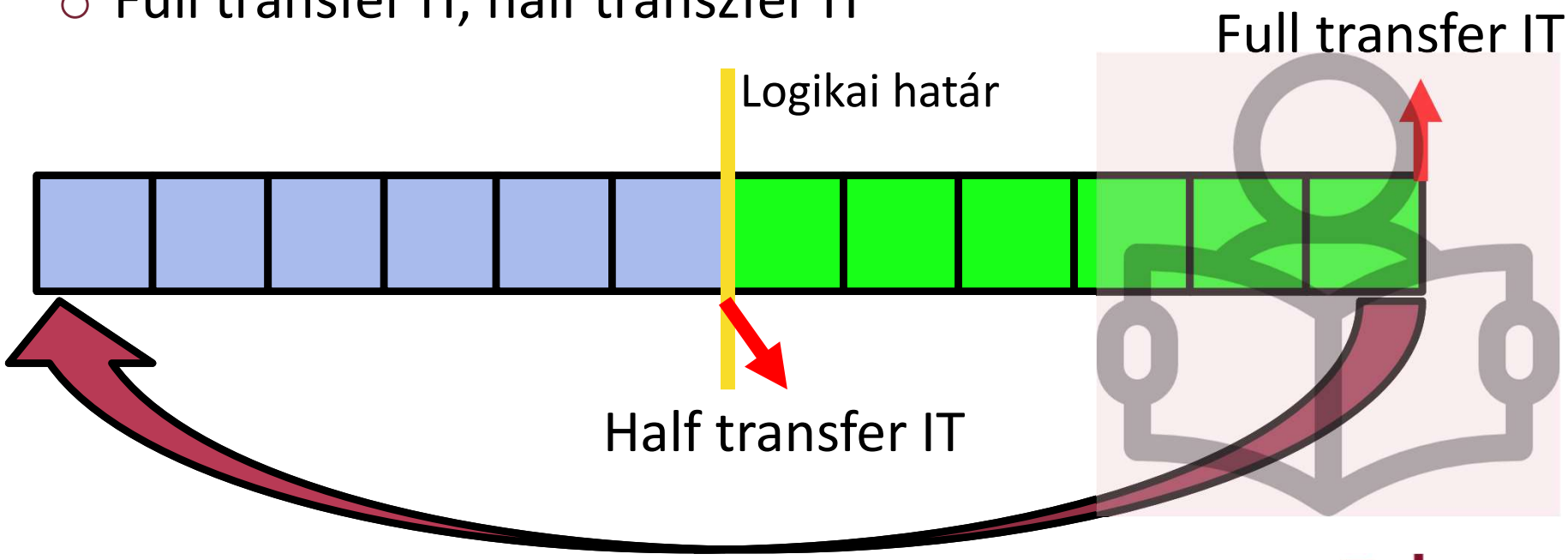
Hogy valósítsuk meg ezt DMA-val?

- **Cirkuláris buffer opció:** amint a végére ért a buffernek a DMA elkezdi az elejéről az írást
- **A hardware-es DMA Write pointert bár általában látható egy regiszterként, de problémás lehet az aszinkron kezelése!**



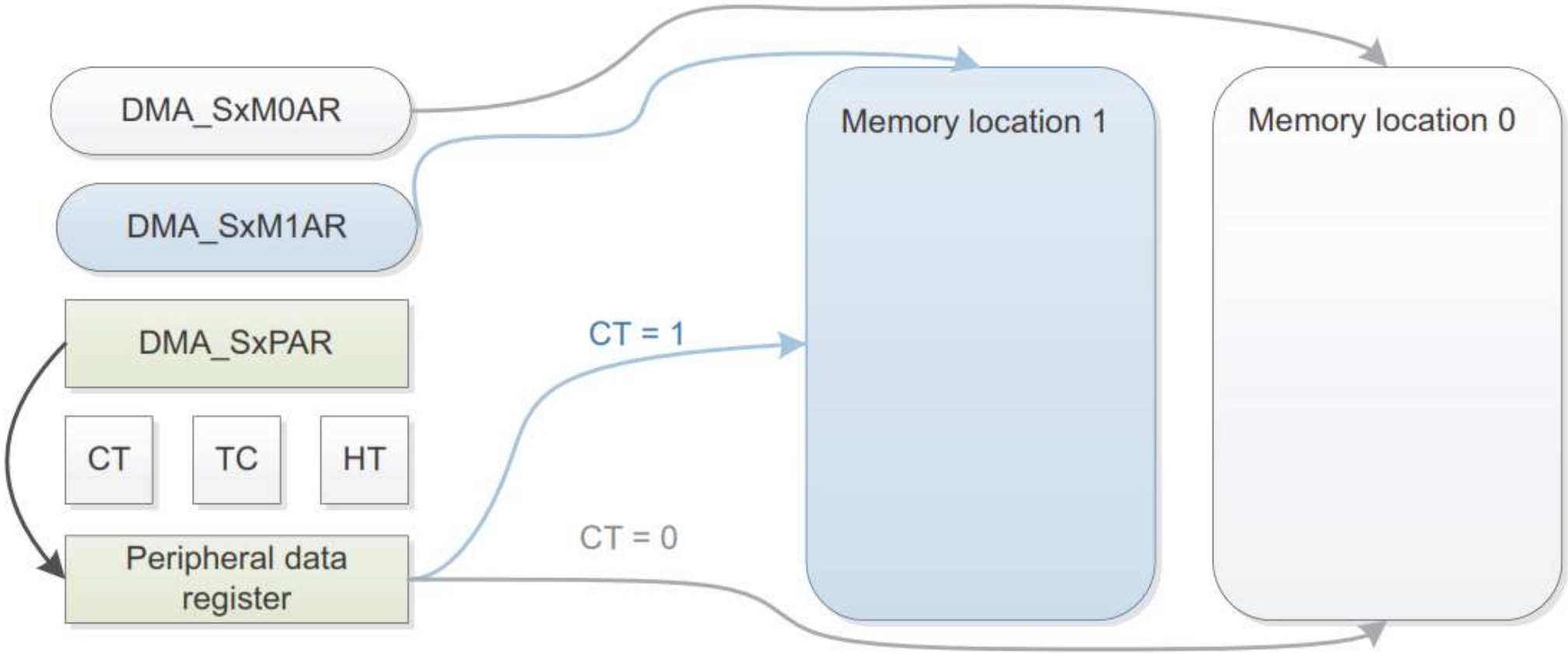
Hogy valósítsuk meg ezt DMA-val?

- **Hogy vesszük észre, hogy melyik részt használhatjuk?**
  - **Az aszinkronitás miatt nem jó általában hardware-es read és write pointer olvasgatni**
  - Két részre kell bontani a buffer, az egyikbe írhat a DMA, a másikból olvashatunk mi (időlimitet a szinkronitás miatt be kell tartani)
  - Full transfer IT, half transzfer IT



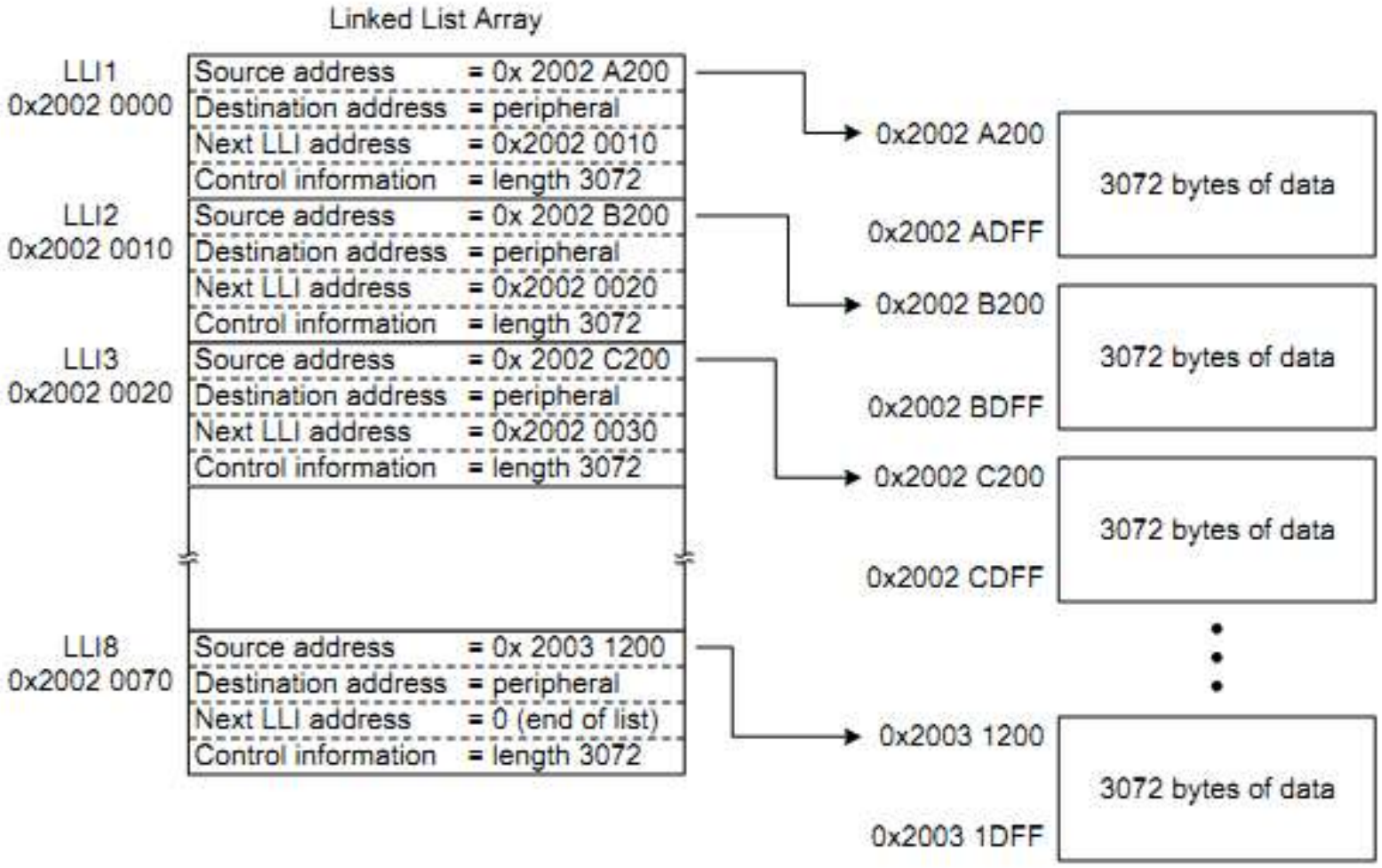
Ugyanez kicsit modernebben

- **Dupla bufferelés, Silabs esetben ping-pong átvitel**



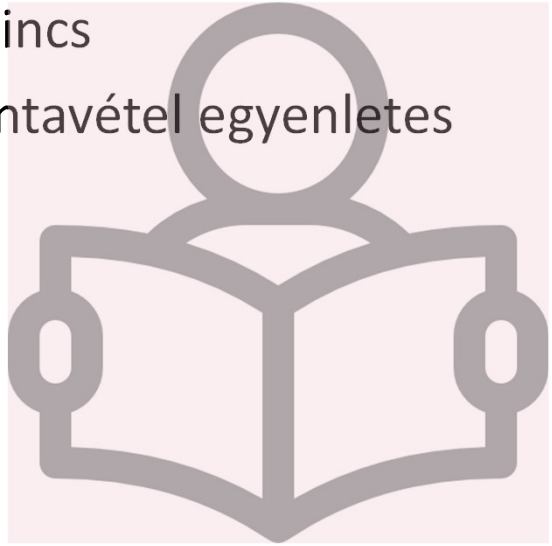
És még bonyolultabban

- **Scatter and gather DMA**
  - Gyakorlatilag az átviendő információ egy linked list-ben átadható memória darabokból áll: pl. kommunikáció üzenet header-payload külön



AD átalakító folyamatos mérés DMA beállítása példa

- **Átvitel forrás báziscíme:** az ADC eredményregisztere
- **Cél báziscíme:** a mintákat tároló (globális!) memória tömb
- **Átviendő blokk hossza:** a buffer mérete mintákban
- **Adatszélesség:** 16 bit (halfword), 12 bites eredményhez is
- **Ciklus végéhez tartozó megszakítás jelzés:** half és full transfer IT
- **Burst-ös, vagy egyciklusú hozzáférés:** egyciklusú, minden konverzió egy kérés
- **Cirkuláris buffer, vagy sima átviteli mód:** cirkuláris (vagy ping-pong)
- **Cél cím inkrementálás:** van; forrás cím inkrementálás: nincs
- **Mintavétel indítása:** hardware-es timer trigger, így a mintavétel egyenletes (szoftveres indításnál jitter)



© BME-MIT 2026

Tartalom

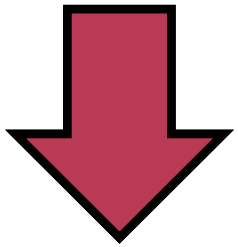
- Az interruptos perifériakezelés korlátai
- DMA alapú perifériakezelés
  - A DMA kezelés alapjai
  - A DMA általános konfigurációs lehetőségei
  - Példák DMA kezelésére
- **Tipikus DMA kezelési hibák. Mit ne csináljunk**
- A DMA helye a rendszer architektúrában
- DMA alapú perifériakezelés korlátai

DMA felprogramozásnál ne csináljuk

- **Periféria cél címnél vagy forrás címnél cím inkrementáció bekapcsolása**
  - Pl.: UART átvitel UART0->TXDATA regiszter címére

DMA felprogramozásnál ne csináljuk

- **Periféria cél címnél vagy forrás címnél cím inkrementáció bekapcsolása**
  - Pl.: UART átvitel UART0->TXDATA regiszter címére



  - Cím inkrementálásnál végigmegy az adatfeltöltés és a következő infót nem az adat regiszterbe írja, hanem valamilyen konfigurációs regiszterbe: nem szándékosan átkonfiguráljuk a perifériákat…



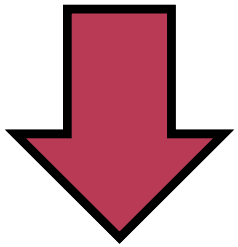
DMA felprogramozásnál ne csináljuk

- **Ne használjuk lokális memóriát forrás, vagy cél címként**
  - Pl.: Egy függvényben felkonfiguráljuk a DMA átvitelt, de a függvény lokális változóját adjuk meg forrásként.

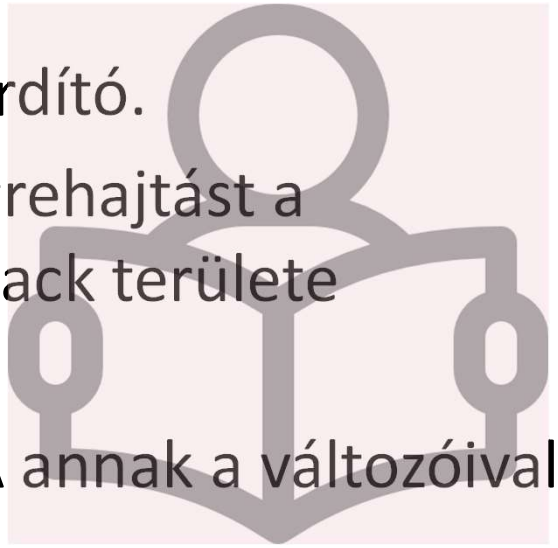
© BME-MIT 2026

DMA felprogramozásnál ne csináljuk

- **Ne használjuk lokális memóriát forrás, vagy cél címként**
  - Pl.: Egy függvényben felkonfiguráljuk a DMA átvitelt, de a függvény lokális változóját adjuk meg forrásként.

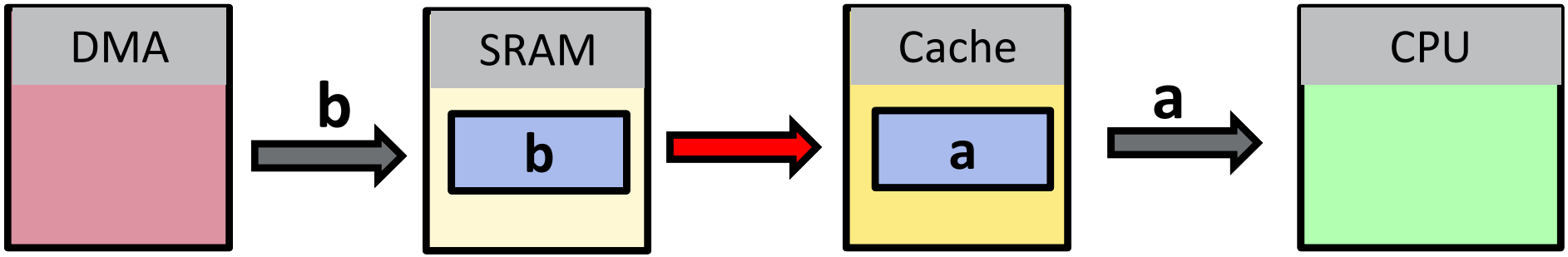


  - Lokális változókat a stack-re foglalja a fordító.
  - Mivel a DMA átvitel nem blokkolja a végrehajtást a függvényből visszatérünk -> Függvény stack területe felszabadul
  - Belépünk egy másik függvénybe -> DMA annak a változóival kerül interakcióba

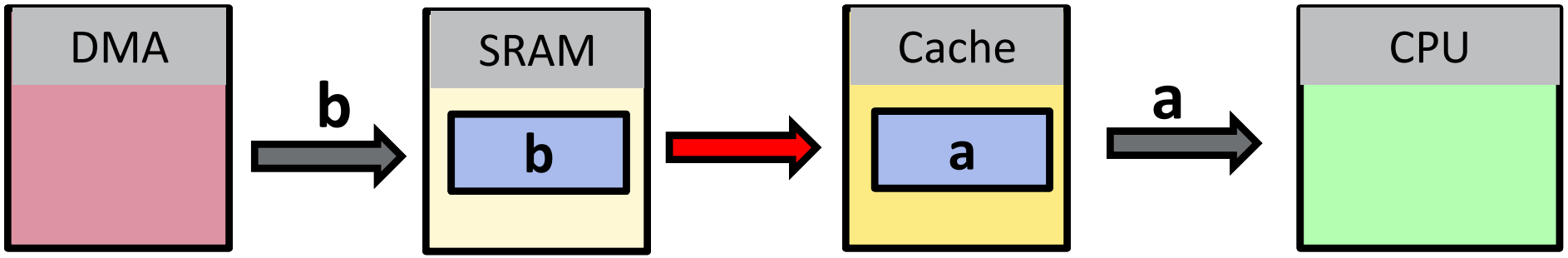


Cache és DMA együttes használata

- Modern mikrovezérlőkben már elképzelhető a Cache megjelenése: Cortex M7, Cortex M55, M85 magok



Cache és DMA együttes használata



- A DMA aktívan masterként használja az adatmemóriát ez koherencia problémát okozhat mindkét átviteli irányban!
- Amennyiben a hardware ezt nem kezeli automatikusan (valószínűleg nem tudja), akkor
  - DMA -> SRAM átvitel előtt és után cache invalidation kell szoftverből végrehajtani
  - SRAM -> DMA átvitelnél pedig cache flush/clean-t kell kérni az átvitel előtt

© BME-MIT 2026

DMA kész ≠ periféria kész

- UART TX DMA complete → az utolsó byte lehet még a UART shift regiszterben.
- SPI TX DMA complete → az SPI lehet még BUSY.
- Flash write DMA complete → a Flash még programoz.
- Kommunikációhoz kapcsolható vonalmeghajtót tartalmazó esetben kimondottan veszélyes lehet: ha DMA complete után azonnal lekapcsoljuk a drivert, az utolsó részt levághatjuk.

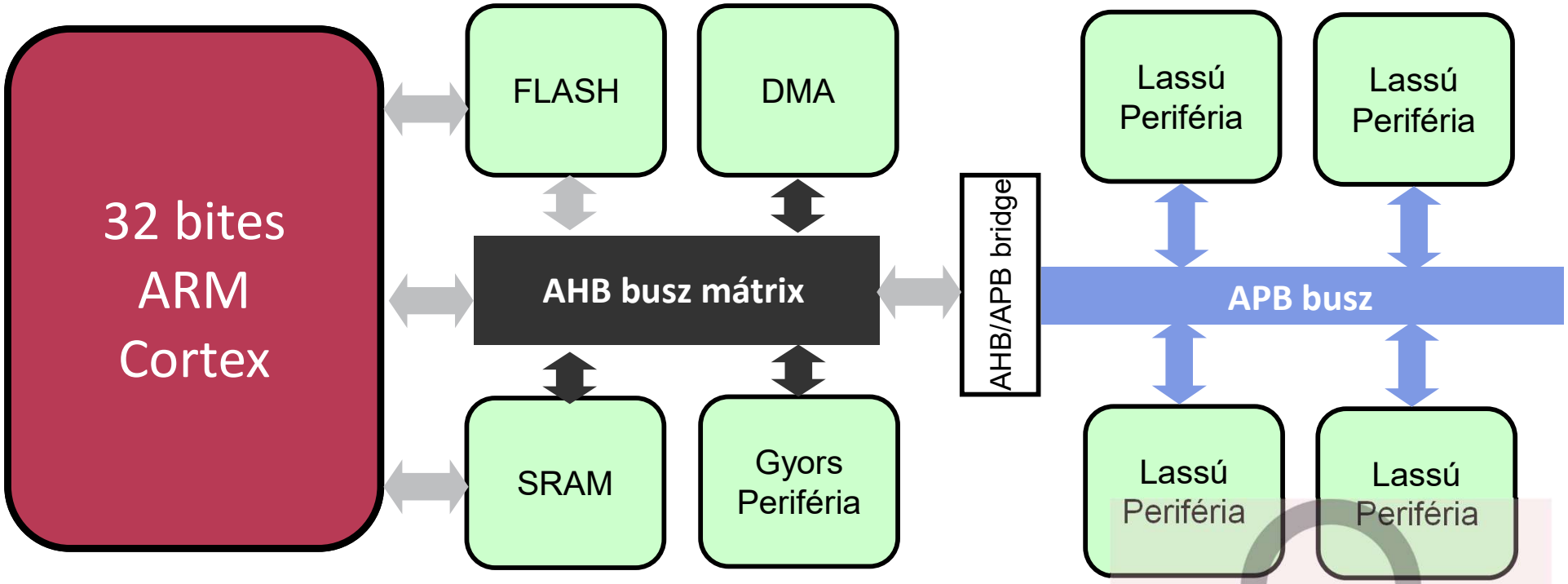


Tartalom

- Az interruptos perifériakezelés korlátai
- DMA alapú perifériakezelés
  - A DMA kezelés alapjai
  - A DMA általános konfigurációs lehetőségei
  - Példák DMA kezelésére
- Tipikus DMA kezelési hibák. Mit ne csináljunk
- **A DMA helye a rendszer architektúrában**
- DMA alapú perifériakezelés korlátai

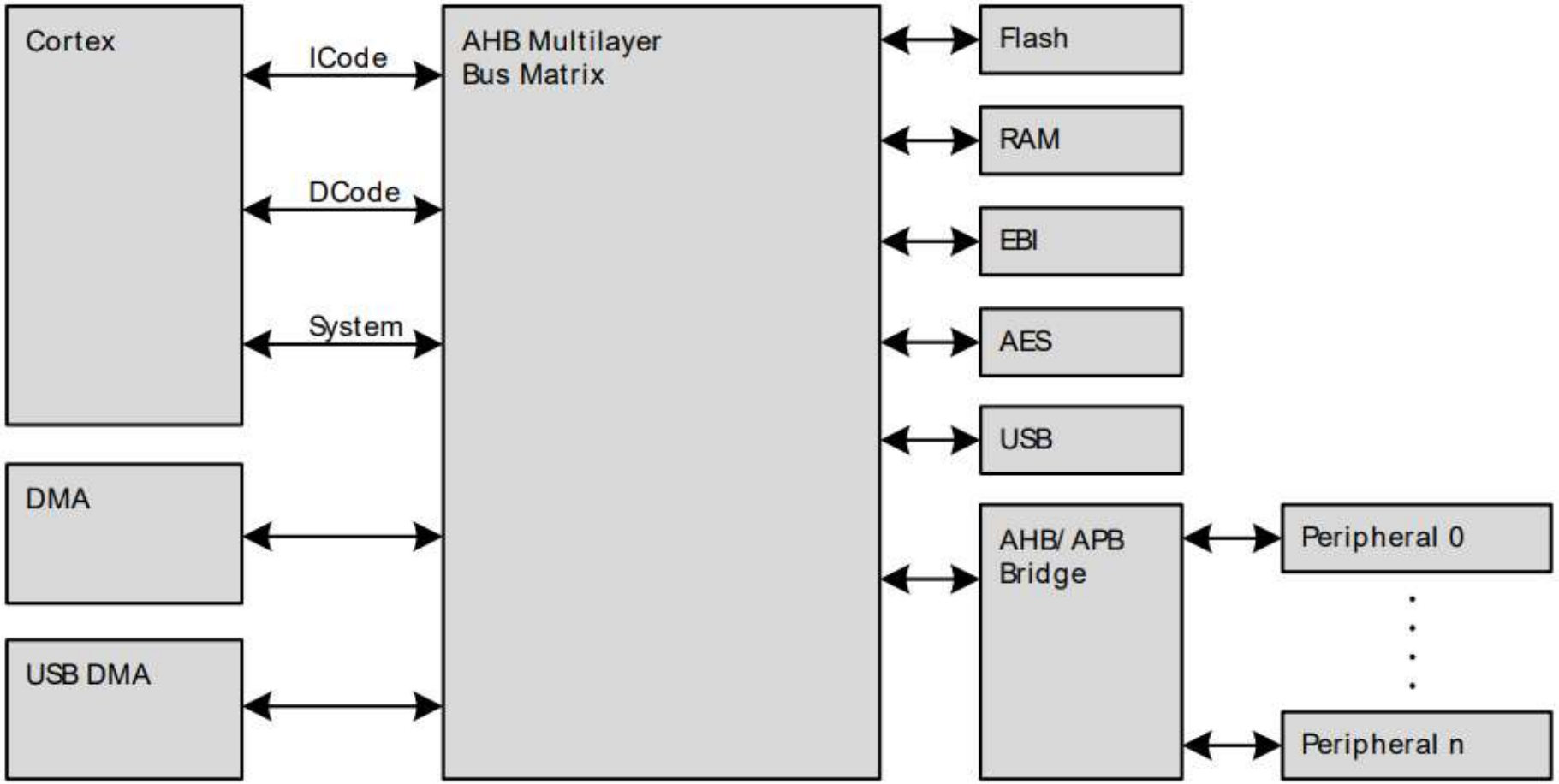
© BME-MIT 2026

32-bites vezérlők belső felépítése



- Megjelenik két különböző képességű busz AHB - APB
- Egyszerűbb esetben (M0, M0+) az AHB mátrix helyett csak egy busz van

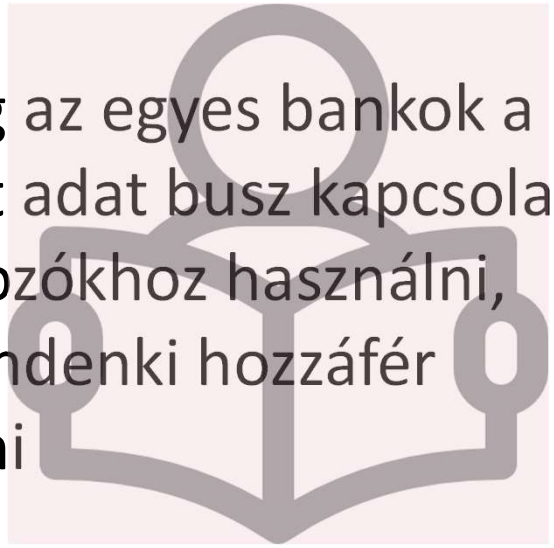
EFM32GG belső felépítése



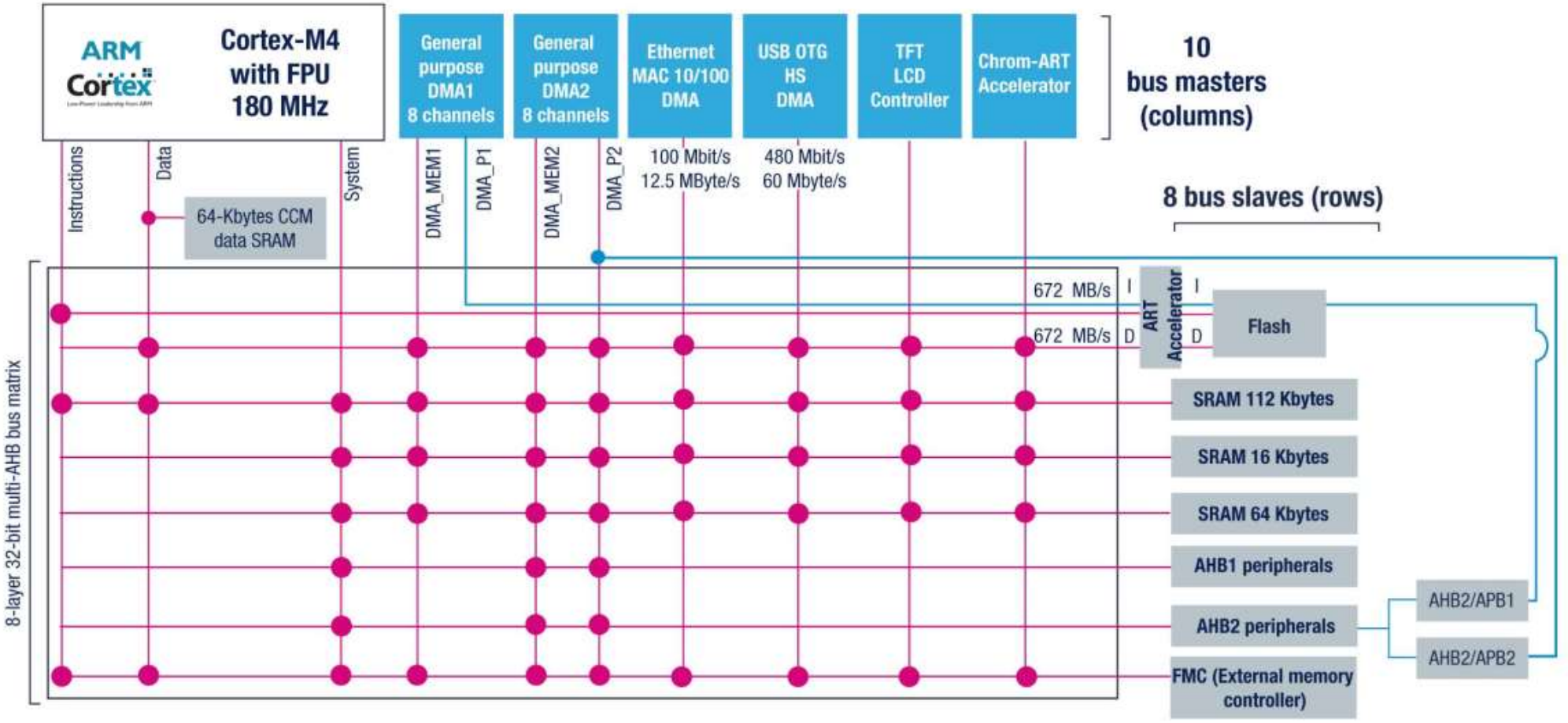
- Megegyezik a mintával
- Hol érzünk ebben egy szűk keresztmetszetet DMA szempontjából, ami miatt jellemzően már ennek egy fejlettebb változatát használják?

Belső felépítés változása az intenzív DMA alkalmazás hatására

- **Az SRAM, mivel mind a CPU mind a DMA átvitelek célpontja/forrása több bankra oszlik, amik párhuzamosan elérhetők**
  - A több bank lehet egymás után folytonos címtartományban, így ezt a programozót nem kell hogy zavarja, ha nincs rá szüksége
  - Ugyanakkor kihasználható az, hogy amíg az egyes bankok a CPU számára vannak prioritizálva (direkt adat busz kapcsolat) és ezért érdemes ezeket a program változókhoz használni, addig a többi bankba amihez szintén mindenki hozzáfér nyugodtan lehet párhuzamosan DMA-zni



Példa egy bonyolultabb vezérlő belső felépítésére: STM32F4xx



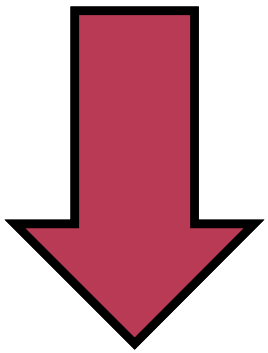
Tartalom

- Az interruptos perifériakezelés korlátai
- DMA alapú perifériakezelés
  - A DMA kezelés alapjai
  - A DMA általános konfigurációs lehetőségei
  - Példák DMA kezelésére
- Tipikus DMA kezelési hibák. Mit ne csináljunk
- A DMA helye a rendszer architektúrában
- **DMA alapú perifériakezelés korlátai**

© BME-MIT 2026

Mikor nem éri meg DMA-t használni

- DMA használat hátránya
  - Macerás felkonfigurálni
  - Könnyű elrontani a felkonfigurálást
  - Az elrontott felkonfigurálást nehéz ki-debuggolni

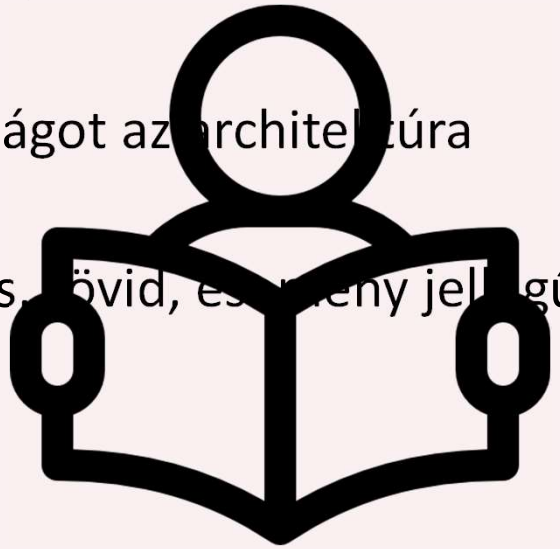




- **Rövid idejű, nem rendszeres, esemény jellegű funkcióra nem érdemes használni**
- **Byte-onkénti döntést igénylő átvitelre sem**

Összefoglalás: DMA alapú perifériakezelés

- A DMA a CPU helyett a háttérben végzi a periféria és a memória közötti adatmozgatást, a CPU-nak csak a felkonfigurálás és az átvitel utáni feldolgozás marad.
- A DMA-t és a perifériát is fel kell konfigurálni: forrás, cél, inkrementálás, adatszélesség, blokkhossz, mód, trigger, jelzések.
- Folyamatos átvitelnél cirkuláris buffer (half/full transfer IT) vagy ping-pong a megoldás, a feldolgozás időlimitjét be kell tartani.
- Tipikus hibák: periféria cím inkrementálása, lokális (stack) buffer, cache koherencia, DMA kész ≠ periféria kész.
- A DMA és a CPU a buszrendszeren osztozik, a hatékonyságot az architektúra (busz mátrix, SRAM bankok) is meghatározza.
- Nagy mennyiségű, rendszeres adatmozgatásnál érdemes, rövid, eseményjellegű vagy byte-onkénti döntést igénylő átvitelnél nem.



© BME-MIT 2026